

25 June, Leetcode POTD

3737. Count Subarrays With Majority Element I

Problem Statement

Given:

```
nums  
target
```

Count the number of subarrays where:

```
target appears strictly more than half  
of the subarray length
```

Example:

```
nums = [1,2,2,3]  
target = 2
```

Valid:

```
[2]  
[2]  
[2,2]  
[1,2,2]  
[2,2,3]
```

Answer:

```
5
```

Step 1: Train Your Intuition

Whenever you see:

```
Count subarrays  
Count substrings  
Number of intervals
```

Immediately think:

```
Can I generate every subarray?
```

because every optimization starts from brute force.

Step 2: Understanding Majority

Majority means:

```
frequency(target) > length/2
```

Example:

```
[2,2,1]  
  
freq = 2  
length = 3  
  
2 > 1.5  
  
Valid
```

Example:

```
[2,1]  
  
freq = 1  
length = 2  
  
1 > 1  
  
False
```

Must be STRICTLY greater.

Step 3: Brute Force Idea

Generate every subarray.

For every subarray:

```
Count occurrences of target
Check majority condition
```

Brute Force Code Template

```
count = 0

for i in range(n):
    freq = 0

    for j in range(i, n):
        if nums[j] == target:
            freq += 1

        length = j - i + 1

        if freq > length // 2:
            count += 1
```

Why Does This Work?

For every start:

```
i
```

we extend:

```
i
i..i+1
```

```
i..i+2  
i..i+3
```

and maintain frequency.

Complexity

Outer loop:

```
O(n)
```

Inner loop:

```
O(n)
```

Total:

```
O(n2)
```

Space:

```
O(1)
```

Now Think Like an Interviewer

Question:

```
What is repeatedly computed?
```

Answer:

```
Frequency of target  
Length
```

Length is easy.

Frequency can be maintained while expanding.

So brute force is already pretty decent.

Important Observation

Condition:

```
freq(target) > length/2
```

Multiply by 2:

```
2 * freq(target) > length
```

Now replace:

```
length = target_count + non_target_count
```

So:

```
2 * target_count >  
target_count + non_target_count
```

Move terms:

```
target_count >  
non_target_count
```

Huge observation.

New Interpretation

Instead of majority:

Think:

```
target_count  
must be greater than  
all other elements combined
```

Example:

```
[2,2,3]  
  
target_count = 2  
others = 1  
  
2 > 1
```

Valid.

Convert Into Prefix Sum Problem

Transform array:

```
target  -> +1  
others  -> -1
```

Example:

```
nums = [1,2,2,3]  
target = 2
```

Convert:

```
[-1,+1,+1,-1]
```

Why?

Suppose subarray:

```
[2,2,3]
```

Converted:

```
[+1,+1,-1]
```

Sum:

```
1
```

Positive.

Meaning:

```
target_count > other_count
```

Exactly our condition.

New Problem

Now becomes:

```
Count subarrays  
whose sum > 0
```

after transformation.

Pattern Recognition

Whenever you see:

```
More X than Y  
Majority  
Balance  
Equal zeros and ones  
More positives than negatives
```

Think:

```
Convert into +1 / -1
```

This is a SUPER common pattern.

Prefix Sum Construction

Example:

```
[-1,+1,+1,-1]
```

Prefix:

```
index : value  
  
0 : 0  
1 : -1  
2 : 0  
3 : 1  
4 : 0
```

Subarray Sum Formula

For subarray:

```
l..r
```

sum:

```
prefix[r+1] - prefix[l]
```

Need:

```
prefix[r+1] > prefix[l]
```

What Are We Counting?

For each current prefix:

How many previous prefixes
are smaller than me?

This becomes a classic counting problem.

Pattern Name

Count pairs

```
prefix[j] > prefix[i]  
where i < j
```

Very similar to:

Count inversions
Count range sums
Count subarrays with positive sum

Optimized Approach

Maintain:

Frequency of previous prefix sums

Count:

How many are smaller
than current prefix

Using:

Fenwick Tree
BIT
Ordered Map
Balanced BST

Why LeetCode Marks It Medium?

Constraint:

$n \leq 1000$

So $O(n^2)$ passes.

The intended solution for Part I is simply:

$O(n^2)$

No need for Fenwick Tree.

Final Accepted Solution

```
class Solution:
    def countSubarrays(self, nums, target):

        n = len(nums)
        ans = 0

        for i in range(n):

            freq = 0

            for j in range(i, n):

                if nums[j] == target:
                    freq += 1

            length = j - i + 1

            if freq > length / 2:
                ans += 1
```

```
return ans
```

Dry Run

Example:

```
nums = [1,2,2,3]
target = 2
```

Start:

```
i = 0
```

Expand:

```
[1]

freq=0
length=1

0 > 0.5

False
```

```
[1,2]

freq=1
length=2

1 > 1

False
```

```
[1,2,2]

freq=2
length=3
```

```
2 > 1.5
```

```
True
```

```
count = 1
```

```
[1,2,2,3]
```

```
freq=2
```

```
length=4
```

```
2 > 2
```

```
False
```

Continue similarly.

Final:

```
5
```

Pattern Identification Cheat Sheet

If problem says:

```
Count subarrays
```

Think:

```
Brute force first
```

```
Prefix Sum second
```

If problem says:

```
Frequency
```

```
Majority
```

More than half

Think:

Can I convert into counts?

If problem says:

More A than B

Think:

A -> +1
B -> -1

If problem says:

Equal A and B

Think:

A -> +1
B -> -1

Need sum = 0

Real Interview Patterns

Pattern 1

Count subarrays with equal 0s and 1s

Transform:

```
0 -> -1  
1 -> +1
```

Need:

```
sum = 0
```

Pattern 2

```
More 1s than 0s
```

Transform:

```
1 -> +1  
0 -> -1
```

Need:

```
sum > 0
```

Pattern 3

```
Majority element
```

Transform:

```
target -> +1  
others -> -1
```

Need:

```
sum > 0
```

Pattern 4

Equal vowels and consonants

Transform:

```
vowel -> +1  
consonant -> -1
```

Need:

```
sum = 0
```

Golden Rule

When stuck on subarray problems:

1. Write brute force
2. Identify condition
3. Convert condition into math
4. Express using prefix sums
5. Look for a known pattern

This process solves nearly 80% of medium-level subarray problems.